

The Runic System

Technical Guide

A plain-language explanation of how the classroom rune-tracking app works, plus the technical reference a future Claude session needs to safely extend it.

Covers: `index.html` (frontend, hosted from GitHub on Render) + `Code.gs` (backend) + Google Sheet backend, including the Students, Pins, Config, Runewords, RuneAwards, and Transforms tabs.

Also documents: `runic-system.html`, the offline single-device fallback version (unchanged by anything in this revision).

Revision note: this edition documents four things added since the last edition — the teacher “Award a Rune” action and its history log, an automatic Transform History log, a reorganized Teacher tab (three panels stay visible by default, the rest collapse behind click-to-open subtabs), and two small pieces of frontend polish (a cross-fading site logo and an 8-bit sparkle effect on rune reveals). It also documents the teacher-replaceable Site Logo feature and its Config keys, which existed in the code but were missing from the previous edition of this guide.

Read this first if you are Claude

This document is written to be pasted into a future conversation as context before making changes to the Runic System app. Section 9 (“Notes for future Claude sessions”) is aimed specifically at you: it lists where each piece of logic lives, what must stay in sync between files, and the deployment steps a change requires.

Skim Sections 2–5 for the architecture and data model before editing anything. The single most important architectural fact carried over from the previous edition: runeword game content (names, recipes, descriptions, photos) lives only in the Google Sheet's Runewords tab, not in either code file. The twelve base runes' stats are still the one exception that stays duplicated between `index.html` and `Code.gs`. See Section 5.4 and Section 9.2 for what that means for content edits.

New in this edition: two more append-only log tabs, RuneAwards and Transforms (Sections 5.5 and 5.6), and a restructured Teacher tab where only three panels are visible by default and the rest live behind collapsible subtabs (Section 6.7). If you're asked to add another Teacher-tab panel, read Section 6.7 first — it explains the pattern to follow so the new panel behaves consistently with the existing ones.

1. What this app is, in plain terms

The Runic System is a classroom reward tool for a Dungeons & Dragons campaign. Students earn runes — twelve different kinds, each with its own rarity — by redeeming one-time codes the teacher hands out at quest milestones. Runes can then be combined into runewords: magic items usable in the tabletop game (a ring, a sword, a pair of boots, and so on), each requiring a specific combination of runes to craft.

The app is a website with three main things a student can do, plus a teacher-only control panel:

- **Roll** — redeem a one-time milestone code for three random runes.
- **Transform** — trade in a combination of lower-value runes to forge a specific higher-value rune, or destroy one of each of the twelve ordinary runes at once to forge a God Rune. Every Transform, of either kind, is now logged for the teacher — see Section 6.6.
- **Craft** — spend a specific set of runes to permanently forge a named magic item (a “runeword”).
- **Teacher tools** — generate milestone codes, manage the entire runeword catalog, award a specific rune directly to a student, see every student's progress, reset a forgotten PIN, and replace the site's header logo. Most of these now live behind a click-to-open subtab rather than one long always-visible scroll — see Section 6.7.

Every student's rune count lives in one shared, always-up-to-date place, so it doesn't matter whether a student checks their runes on a classroom Chromebook, their own phone, or the teacher's laptop — everyone sees the same numbers. The same is true of the runeword catalog, and now also of the rune-award and transform history logs: anything the teacher or a student does from any device is visible everywhere else immediately.

2. How the pieces fit together

There is no traditional “app server.” Instead, three ordinary pieces of infrastructure work together:

Piece	What it is	Where it lives
Frontend	A single self-contained HTML file (index.html) with all its CSS and JavaScript inline, plus an images/ folder of PNGs for seeded runewords. This is what a student's browser actually loads and displays.	A GitHub repository, deployed as a static site on Render (visiirunes.onrender.com). Pushing to GitHub auto-redeploys.
Backend	A small program (Google Apps Script, Code.gs) that receives requests from the frontend, applies the game rules, and reads/writes the spreadsheet.	Runs inside Google's infrastructure, attached to a Google Sheet.
Database	An ordinary Google Sheet with six tabs (Students, Pins, Config, Runewords, RuneAwards, Transforms) used as a simple row-based database.	Lives in the teacher's Google Drive.

The request flow for, say, rolling for runes, looks like this:

Student's browser	Google Apps Script	Google Sheet
-----	-----	-----
1. Click "Roll"		
2. fetch() POST ----->	2. doPost() receives it	
3. Checks the student's PIN -->	reads Students tab	

4. Checks the milestone code -> reads Pins tab
5. Rolls 3 random runes
6. Writes new totals -----> writes Students tab
7. Marks the code as used ----> writes Pins tab
8. Shows the 3 runes <----- Returns JSON result

Every single action (rolling, transforming, crafting, generating a code, adding or editing an item, awarding a rune) follows this same shape: the browser sends a small JSON message describing what it wants to do, the Apps Script backend is the only thing that actually checks the rules and touches the spreadsheet, and it sends back the correct, updated data. The browser never edits the spreadsheet directly and never decides on its own whether an action is legal — it just displays whatever the backend says is true.

2.1 Why it's built this way (not, say, a normal database)

- **No server to pay for or maintain.** Google Apps Script and Google Sheets are free, and Google handles hosting, uptime, and security patching. The static frontend on Render's free tier is likewise free to host.
- **The teacher already owns and can read the data at any time,** just by opening the Sheet — no separate admin tool needed to inspect or manually fix something, including individual runeword rows, or the new award/transform history rows.
- **Good enough concurrency for a classroom.** A short server-side lock (Section 5.7) makes sure two people acting at the exact same instant — including a student transforming while the teacher awards them a rune — can't corrupt each other's data.

3. The three kinds of credentials

It's easy to confuse these three, since they're all short numeric codes (or, for the teacher, a short passphrase). They are completely separate and serve different purposes:

Credential	Length	Who has it	What it's for	Where it's checked
Milestone code	6 digits	The whole class shares codes handed out by the teacher	One-time reward: redeeming it grants 3 random runes, then it's burned	Pins tab (Used column flips to TRUE)
Adventurer PIN	4 digits	Each student sets their own, for their own name	Proves "this is really my account" before rolling, transforming, or crafting	Students tab (PinHash column, salted & hashed)
Teacher passphrase	Any length ≥ 3	The teacher only, shared across the whole deployment	Unlocks the Teacher tab: generating codes, managing the item catalog, awarding runes, resetting a PIN	Config tab (TeacherPass row)

The adventurer PIN was built specifically so students can't tamper with each other's runes just by typing a classmate's name. The first time any name is used to roll, transform, or craft, the app asks that person to choose a 4-digit PIN — from then on, that name always asks for it again (once per browser session, since it's cached briefly so students aren't re-typing it constantly). Viewing a name's inventory does not require the PIN; only the three actions that change rune counts do.

Note on "Grant a God Rune": this teacher-only action from earlier revisions no longer exists as its own thing. God Runes are earned entirely by students via Transform (Section 6.1) — though a teacher can now also hand one to a student directly through the general-purpose Award a Rune action (Section 6.5), which supersedes the old single-purpose action.

4. Glossary, for anyone non-technical

Term	Plain-language meaning
Frontend	The part of the app you actually see and click on — the webpage itself.
Backend	The part running behind the scenes that enforces the rules and stores data. Students never see it directly.
API / endpoint	A specific "request type" the backend understands, like "roll runes" or "award a rune." The frontend sends one of these every time it needs something done.
Google Apps Script	A free tool from Google that lets you write small programs that live inside a Google Sheet or Doc and can be reached over the internet.
Deployment	A published, stable version of the backend program with its own web address. Editing the code does nothing to the live app until you deploy a new version.

Term	Plain-language meaning
Hash (of a PIN)	A one-way scrambled version of a PIN. The real 4-digit number is never stored or sent back anywhere — only this scrambled fingerprint, which can confirm a guess is correct without ever revealing the original number.
Lock (concurrency lock)	A brief “hold on, someone else is mid-edit” pause the backend uses so two people acting at the same instant can't corrupt each other's data.
Runeword catalog / Runewords tab	The live, editable list of every craftable magic item, stored as its own tab in the Google Sheet. The teacher edits it from the app; nobody edits code to add an item anymore.
Award / RuneAwards tab	A one-off rune the teacher hands a student directly, outside the normal Roll/Power Word/Transform flow. Every award is logged as its own row on the RuneAwards tab.
Transform log / Transforms tab	An automatic record of every rune Transform any student performs (ordinary or a forged God Rune), one row per Transform, written by the backend itself — no one has to press anything extra for a Transform to be logged.
Subtab	A secondary tab strip inside the Teacher tab. Clicking a subtab opens its panel (and fetches its data for the first time); clicking it again collapses it. See Section 6.7.
Seed / seeding	A one-time “fill in the starting data” step. The Runewords tab is seeded once, the first time it's empty, with a starter catalog — after that, the sheet (not the code) is the source of truth.
localStorage / sessionStorage	Small amounts of data a website can remember inside one specific browser. Used here for convenience settings and the baked-in backend URL override, never for the actual rune, item, or history data.

5. The data model (the Google Sheet)

The Sheet has six tabs. Each row in Students, Pins, Runewords, RuneAwards, or Transforms is one record, exactly like a row in any spreadsheet.

5.1 Students tab — one row per adventurer

Column	Contains
Name	The adventurer's name, exactly as typed. Matching is case-insensitive.
TIR, ITH, NEF, TAL, ORT, THUL, SHAEL, IO, LUM, JAH, BER, ZOD	One column per ordinary rune, holding how many of that rune this student currently owns.
GOD	How many God Runes this student owns — forged via Transform, or handed over directly by the teacher via Award a Rune.
Crafted	A JSON list of every runeword this student has forged, each with a name and timestamp. e.g. <code>[{"name":"Enigma","at":"2026-09-18T..."}]</code>
PinHash	A salted SHA-256 fingerprint of the student's 4-digit PIN. Never the PIN itself.
PinSalt	A random value mixed into the PIN before hashing, unique per student, so two students with the same PIN don't produce the same hash.
PinFailCount	How many wrong PIN attempts in a row. Resets to 0 on a correct guess.
PinLockUntil	If set, a timestamp before which this student's PIN is temporarily locked out (after 5 wrong attempts in a row).

5.2 Pins tab — one row per milestone code ever generated

Column	Contains
Code	The 6-digit one-time code.
Used	TRUE once someone has redeemed it, otherwise FALSE.
UsedBy	Which adventurer name redeemed it.
CreatedAt / UsedAt	Timestamps for when the teacher generated it and when it was redeemed.

5.3 Config tab — small key/value settings

Holds a handful of standalone key/value rows. This edition documents two keys (LogoImage, LogoUpdatedAt) that existed in the code but were missing from the previous edition of this guide:

Key	Value holds
TeacherPass	The shared teacher passphrase, set the first time anyone unlocks the Teacher tab.

Key	Value holds
LogoImage	Either empty (the site shows its default rune emblem) or a base64 data: URI for the teacher-uploaded header logo. See Section 6.3.
LogoUpdatedAt	Timestamp of the last time LogoImage was set or cleared. Currently informational only — not read by any game rule.

5.4 Runewords tab — one row per craftable item — the single source of truth

This tab replaces the old hardcoded RUNEWORDS arrays that used to live, duplicated, in both index.html and Code.gs. It is created and seeded once by the setup function (see Section 9.1) from the RUNEWORD_SEED constant in Code.gs, and is never re-seeded after that — from that point on, every add, edit, hide, or delete happens directly on this tab, live, from the Teacher tab's "Manage Items" panel (now one click away behind a subtab, see Section 6.7), with no code change or redeploy needed.

Column	Contains
Id	A URL-safe slug (e.g. "enigma"). Auto-generated from the name when an item is added; kept unique by appending -2, -3, etc. if needed.
Name	Display name of the item, e.g. "Enigma."
Slot	Equipment slot / category, e.g. "Chest Armor," "Ring," "Food."
Recipe	A JSON array of rune codes required, e.g. ["BER","NEF","THUL","ORT"]. Can include "GOD" as an ingredient (used by Summarmækir and Ran's Net).
Effect	Free-text description of the item's in-game effect.
Quote	Optional flavor-text quote.
Image	Either blank (falls back to images/{id}.png shipped in the frontend's images/ folder) or a base64 data: URI the teacher uploaded through the item form, client-compressed before upload — see Section 6.2.
Hidden	TRUE/FALSE. Hidden items are excluded from the student-facing Craft grid and Codex, and can't be crafted, but stay in the catalog for the teacher to unhide later.
UpdatedAt	Timestamp of the last add/edit/hide-toggle on this row.

What's still duplicated, and what isn't: the twelve ordinary runes' codes, roll odds, and point values (the RUNES constant — Section 7.1) are still hand-kept in sync between index.html and Code.gs, because they're simple and change rarely. Runeword content lives only in the Sheet, is fetched live by both the student view (getRunewords) and the teacher view (teacherListRunewords), and must never be hardcoded back into either file.

5.5 RuneAwards tab (new) — one row per teacher-awarded rune

Every time the teacher uses the “Award a Rune” action (Section 6.5), one row is appended here. This tab is written to only by `apiTeacherAwardRune` in `Code.gs`; nothing else appends to it, and nothing on the student side reads it — it exists purely as a teacher-visible history log.

Column	Contains
Name	The adventurer who received the rune, exactly as the teacher selected it from the dropdown.
Rune	The rune code awarded — one of the twelve ordinary codes, or GOD.
At	ISO timestamp of when the award happened.

The Teacher tab's “Rune Award History” table reads this tab back via `teacherListRuneAwards`, newest row first.

5.6 Transforms tab (new) — one row per Transform any student performs

Unlike `RuneAwards`, nobody has to press an extra button for a row to land here: `apiTransform` in `Code.gs` appends one row every time a Transform succeeds, whether it's an ordinary point-value trade or a God Rune forge (Section 6.1). This makes the log complete by construction — there's no separate “log this Transform” step that a future code change could accidentally skip if it's built into `apiTransform` itself, but see the caution in Section 9.2 about keeping it that way.

Column	Contains
Name	The adventurer who performed the Transform.
Target	The rune code they forged — one of the twelve ordinary codes, or GOD.
Destroyed	A JSON object of exactly which runes were spent to pay for it, as <code>{CODE: quantity}</code> , e.g. <code>{"TIR":6,"ITH":4}</code> . For a God Rune forge this is always all twelve ordinary codes at quantity 1.
At	ISO timestamp of when the Transform happened.

The Teacher tab's “Transform History” table reads this back via `teacherListTransforms`, newest row first, rendering the `Destroyed` object as a readable string like “6× TIR, 4× ITH.”

5.7 How concurrency is handled

Every backend action that changes data (rolling, transforming, crafting, generating a code, adding/editing/hiding/deleting an item, awarding a rune, etc.) wraps its work in a `LockService` lock. If two people happen to click at the exact same moment — including a student transforming a rune the teacher is simultaneously awarding them one of, or a student crafting an item the teacher is simultaneously editing — the second request simply waits its turn (up to 10 seconds) rather than the two writes colliding and corrupting data.

6. How the newer features actually work

6.1 The God Rune formula

A student forges a God Rune themselves, from the Transform tab, by choosing GOD as the target rune. Unlike an ordinary Transform (which trades any combination of runes worth enough total points), forging a God Rune uses a fixed recipe, not a point trade: the student must destroy exactly one of each of the twelve ordinary runes (Tir, Ith, Nef, Tal, Ort, Thul, Shael, Io, Lum, Jah, Ber, Zod) in a single action, which yields exactly one God Rune. Owning extra copies of a rune beyond the one required doesn't help — the backend (apiTransform in Code.gs) checks the target is “GOD” first and, if so, requires a quantity of at least 1 for every single code in RUNE_CODES before allowing the trade. Two runewords (Summarmækir and Ran's Net) each require one God Rune in their own recipe.

This is validated entirely server-side, in the same function that handles ordinary Transforms — there's no separate API action for it, and that function is also where every Transform now gets logged to the Transforms tab (Section 5.6). If you ever change this recipe, the change belongs in apiTransform in Code.gs, and nowhere in the frontend, since the frontend never decides whether a trade is legal.

6.2 Manage Items: add / edit / hide / delete runewords

The Teacher tab's “Manage Items” panel (now behind a click-to-open subtab, see Section 6.7) lists every runeword, including hidden ones, with Edit, Hide/Show, and Delete buttons, plus an “+ Add New Item” button that opens the same form used for editing. The form collects a name, slot, description/effect, an optional flavor quote, an optional photo, and the recipe (clicking rune icons to toggle them on/off, GOD included).

Action	Frontend call	What the backend does
Add	teacherAddRuneword	Validates the name, slot, and recipe; slugifies the name into a unique Id; appends a new row to Runewords with Hidden = false.
Edit	teacherEditRuneword	Finds the row by Id; updates only the fields actually sent; re-validates the recipe and image size if those changed; stamps UpdatedAt.
Hide / Show	teacherSetRunewordHidden	Flips the Hidden column for that row. Hidden items vanish from Craft and the Codex immediately, but the row is untouched.
Delete	teacherDeleteRuneword	Permanently removes the row after a confirm() prompt. Students who already crafted that item keep it in their own Crafted list, but nobody can craft it again.

Implementation note: the recipe-toggle rune icons in the item form are <label> elements wrapping a hidden checkbox. The toggle logic listens for the checkbox's **change** event, not a click on the label. If a future edit moves that listener back onto the label's own click event, clicking a rune icon will silently do nothing — a label click also fires a synthetic click on its wrapped checkbox, which bubbles back through the label and re-triggers the same listener a second time, toggling the class on and then immediately back off.

6.3 Site Logo (teacher-replaceable header emblem)

NEW TO THIS GUIDE (feature already existed in the code)

The header shows a default hand-drawn rune emblem (an inline SVG) unless the teacher has uploaded a custom logo image, in which case every visitor sees that image instead. The image is stored directly in Config

(LogoImage, as a base64 data: URI) rather than its own sheet tab, since it's a single global setting rather than a per-row record.

Action	Frontend call	What the backend does
View	getLogo	Returns the current LogoImage value (or empty string if none is set). Called by every visitor on page load.
Upload / clear	teacherSetLogo	Validates the image (re-checks size server-side, same MAX_IMAGE_CHARS pattern as runeword photos), writes it to Config's LogoImage, and stamps LogoUpdatedAt. An empty image clears back to the default rune.

On the frontend, the default rune SVG and the fetched `<img>` are stacked in the same box. Only the `img`'s opacity is animated (a 2-second CSS transition) once `getLogo()` resolves, so a custom logo cross-fades in over the default rune rather than popping in instantly. Once the fade-in finishes, the SVG is hidden outright (not just visually covered) — this matters because a logo with a transparent background would otherwise show the rune's outline bleeding through behind it permanently. On reset, the SVG is revealed again before the `img` fades back out, so the reveal still cross-fades rather than popping.

6.4 Rune reveal animation: fade-in + 8-bit sparkle burst

When a Roll or a Power Word redemption succeeds, each resulting rune card no longer just appears instantly. It pops in using a CSS `steps()` timing function (rather than a smooth ease), which gives it a deliberately chunky, retro-game feel, and a small burst of blocky, non-anti-aliased pixel “+” sparkles — drawn as a tiny hand-built SVG with `shape-rendering="crispEdges,"` not an image file — flies outward from the rune the instant it appears, in a handful of directions with a small retro color palette. For a 3-rune Roll, the three cards cascade in one after another (staggered roughly 140ms apart) rather than all popping at once. This is purely cosmetic front-end presentation; it has no effect on what runes are actually rolled or how they're validated.

6.5 Award a Rune

NEW

A teacher can hand any single rune — one of the twelve ordinary codes, or GOD — directly to a named adventurer, outside the normal Roll/Power Word/Transform flow. This is meant for one-off rewards that don't fit a milestone code, e.g. an in-session bonus. It deliberately generalizes (and supersedes) the old single-purpose “grant a God Rune” action from an earlier revision, which was removed once students could forge God Runes themselves.

Action	Frontend call	What the backend does
Award	teacherAwardRune	Validates the rune code, creates the student's row if it doesn't already exist (same as any other action that touches a new name), adds one to that rune in their inventory, and appends a row to RuneAwards.

The teacher picks the adventurer from a dropdown (populated from `teacherListStudents`, i.e. only names that already exist in the Students sheet) and the rune from a dropdown listing all twelve ordinary runes plus GOD. After a successful award, the Rune Award History table underneath refreshes automatically.

6.6 Transform History

NEW

Every Transform any adventurer performs — ordinary or a God Rune forge — is now logged automatically to the Transforms tab as part of `apiTransform` itself (Section 5.6), and the Teacher tab's "Transform History" panel lists them via `teacherListTransforms`, newest first, showing the adventurer's name, the target rune, exactly which runes (and how many of each) were destroyed to pay for it, and the timestamp.

6.7 Teacher tab layout: default panels vs. collapsible subtabs

NEW

The Teacher tab used to be one long scroll of every panel at once. It's now split into two groups:

- **Always visible** the moment the Teacher tab unlocks: Generate a Milestone Code, Set a Power Word, and Site Logo.
- **Collapsed behind a subtab strip** until clicked open: Manage Items, Award Runes (which also contains the Rune Award History table), Transform History, Students, PIN History, and Danger Zone.

The subtab strip (`#teacherSubtabs` in `index.html`) behaves like a single-open accordion: clicking a subtab button closes whichever other one was open and opens this one; clicking the currently-open subtab again collapses everything back to nothing showing. Nothing is fetched from the backend for a collapsed subtab — each one's data loads the first time it's actually clicked open, via a small lookup table in the frontend JS (`SUBTAB_LOADERS`), which maps each subtab's key to the render function that fetches and draws its panel. `renderTeacher()` itself, called whenever the Teacher tab needs a general refresh, now only re-renders the three always-visible panels; it no longer eagerly calls the students/pins/items loaders the way it used to.

If you're asked to add another Teacher-tab panel in the future: decide whether it belongs in the always-visible group (rare — reserve this for things a teacher does constantly) or the collapsed group (the default assumption). For the collapsed group, add a `<button data-subtab="yourkey">` to `#teacherSubtabs`, wrap the panel's markup in a `<div class="subtabpane" id="subtab-yourkey" style="display:none">`, and add an entry to `SUBTAB_LOADERS` pointing at whatever function fetches and renders it. That's the entire pattern — the click handler, the open/close bookkeeping, and the single-open-at-a-time behavior are all generic and don't need touching.

Also note: because the subtabs are collapsed by default, a teacher who unlocks the Teacher tab, does nothing else, and locks it again will not have triggered any of the six lazy-loaded panels' backend calls at all — only Generate Milestone Code's status check, Set a Power Word's status check, and Site Logo's `getLogo()` run automatically.

7. Game data reference

Important framing: the runeword table below is the initial seed — what RUNEWORD_SEED in Code.gs puts into the Runewords sheet the very first time setup runs, if the sheet is empty. It is a historical snapshot, not a live listing. The actual current catalog can differ and should always be read from the Runewords sheet itself, or the app's Codex tab, not from this document. The same is true in spirit of the RuneAwards and Transforms tabs (Sections 5.5, 5.6): they have no seed data at all — they start empty and only ever grow from real teacher/student actions.

7.1 The twelve runes (still hardcoded, still duplicated — see Section 5.4)

Rune	Roll odds	Point value (for Transform)
Tir	14%	2
Ith	14%	2
Nef	12%	5
Tal	12%	5
Ort	10%	9
Thul	10%	9
Shael	8%	14
Io	8%	14
Lum	4%	20
Jah	4%	20
Ber	2%	27
Zod	2%	27

The God Rune is a thirteenth, separate kind: it is never rolled (its odds aren't part of the 100% above) and is only obtainable via a student forging it through Transform (Section 6.1) or a teacher handing one over directly through Award a Rune (Section 6.5).

7.2 How a roll is weighted

A roll draws a random number between 0 and 100, then walks down the rune list subtracting each rune's percentage until the number goes negative — whichever rune it lands on is the result. This happens three independent times per Roll action, so duplicates are possible.

7.3 Transform rule (ordinary target)

To forge one ordinary target rune, a student destroys any combination of runes from their own inventory whose point values (Section 7.1) add up to at least the target rune's own point value. Left-over value is not returned. (Targeting GOD instead follows the fixed-recipe rule in Section 6.1, not this one.) Whichever runes are actually spent are recorded verbatim in the Transforms tab — see Section 5.6.

7.4 Runeword recipes — initial seed (26 items + Sour Key)

Runeword	Slot	Recipe
Enigma	Chest Armor	Ber + Nef + Thul + Ort
Cling	Gloves	Io + Nef + Ith
Prism	Necklace	Tir + Nef + Tal
Silver Tongue of Rizz	Eyewear	Shael + Io
Quicksilver	Harness / Bridle	Lum + Ort + Ith
Charger	Harness / Bridle	Ber + Thul + Tal + Shael
Precious	Ring	Zod + Lum + Thul
Hive	Boots	Ber + Lum + Ort + Ith
Starboard	Oar / Paddle	Io + Nef + Tir
Blade of Carotene	Sword	Io + Nef + Tir
Folly	Scarf	Zod + Shael + Ort
Insolence	Weapon	Lum + Shael + Ort + Tal
Sage	Earrings	Tir + Nef + Ort + Shael
Lark	Scarf	Ith + Tal + Thul + Io
Inferno	Dish	Zod + Io + Thul
Socks of Blocks	Socks	Zod + Jah + Io + Tal
Invigorated by the North	Pendant / Necklace	Lum + Shael + Tal
Bramble	Leg Armor	Shael + Ort + Nef + Tir
Crescent Moon	Musical Instrument	Ber + Shael + Tal + Ith
Harmony	Head Armor	Lum + Io + Tal + Tir
Summarmækir	Weapon	1 God Rune + Zod + Jah + Io
Ran's Net	Weapon	1 God Rune + Ber + Lum + Shael
Herofeller	Scabbard	Tir + Tal + Ort + Io
Coolth	Pillow	Tir + Thul + Shael + Lum
Amulet of Misfortune	Necklace	Tir + Shael + Thul + Nef
Stuffed Buddy	Doll	Ith + Nef + Thul + Io
Sour Key	Food	Shael + Io + Jah + Ber, ×1 (Small) / ×2 (Medium) / ×3 (Large)

Sour Key is handled by its own dedicated action (craftSour) rather than living as a normal Runewords row, since it scales by a size multiplier rather than being a single fixed recipe — unchanged from previous revisions and still hardcoded in Code.gs's SOUR_KEY_BASE constant, not editable from Manage Items.

8. Full API reference

The frontend talks to the backend with a single web address (the Apps Script deployment URL, baked into index.html as a default and overridable via the gear-icon settings). Every request is an HTTP POST with a JSON body containing an action field and whatever parameters that action needs. Every response is JSON shaped as either {"ok":true,"data":...} or {"ok":false,"error":"..."}

action	Parameters	What it does	PIN req'd?
checkName	name	Reports whether this name already has a PIN set.	No
claimPin	name, authPin	Sets a brand-new 4-digit PIN for a name that doesn't have one yet.	No
verifyPin	name, authPin	Checks a PIN is correct, without doing anything else.	—
getStudent	name	Returns a student's current inventory and crafted-items list. View only.	No
roll	name, code, authPin	Redeems a 6-digit milestone code for 3 random runes.	Yes
transform	name, target, selection, authPin	Destroys the chosen runes to forge one target rune (or a God Rune via the fixed recipe). Also logs the Transform to the Transforms tab.	Yes
craft	name, itemId, authPin	Destroys a runeword's recipe runes and adds the item to Crafted.	Yes
craftSour	name, mult, authPin	Same as craft, but for the size-scaling Sour Key (mult is 1, 2, or 3).	Yes
getRunewords	—	Returns every visible (non-hidden) runeword. Called on load to populate Craft and Codex.	No
getLogo	—	New (documented). Returns the current site logo image, or empty if none is set.	No
teacherSetup	pass	Sets the shared teacher passphrase (only works once, when none is set).	—
teacherLogin	pass	Checks the teacher passphrase is correct.	—
teacherGeneratePin	pass	Creates a new, unused 6-digit milestone code.	—
teacherListRunewords	pass	Returns every runeword, including hidden ones, for the Manage Items panel.	—

action	Parameters	What it does	PIN req'd?
teacherAddRuneword	pass, name, slot, recipe, effect, quote, image	Validates and appends a new runeword row.	—
teacherEditRuneword	pass, id, (any of: name, slot, recipe, effect, quote, image)	Updates only the fields provided on an existing runeword row.	—
teacherSetRunewordHidden	pass, id, hidden	Shows or hides a runeword from students without deleting it.	—
teacherDeleteRuneword	pass, id	Permanently removes a runeword row. Irreversible.	—
teacherSetLogo	pass, image	New (documented). Validates and stores (or clears) the site's header logo image.	—
teacherAwardRune	pass, name, code	New. Adds one of the given rune to name's inventory (creating the student row if needed) and logs the award.	—
teacherListRuneAwards	pass	New. Returns the full Rune Award History, newest first.	—
teacherListTransforms	pass	New. Returns the full Transform History, newest first.	—
teacherListStudents	pass	Returns every student's totals, crafted count, and whether a PIN is set.	—
teacherResetPin	pass, name	Clears a student's PIN so they can set a new one.	—
teacherReset	pass	Wipes all student data, milestone codes, the Power Word and its redemptions, and now also the RuneAwards and Transforms logs. Irreversible. (Does not touch Runewords.)	—

Removed in an earlier revision: teacherGrantGod (pass, name) no longer exists as a dedicated action. Its purpose is now covered by the general-purpose teacherAwardRune, which can hand out GOD (or any ordinary rune) to a named student.

All “teacher...” actions require the correct pass value in place of a student PIN; there is no separate teacher session token — the passphrase is simply re-sent with every teacher request.

9. Notes for future Claude sessions

If you (a future instance of Claude) are being asked to modify this app, here is what you need to know before touching anything.

9.1 Files and where they live

File	Role	How to change it live
index.html	The real, multi-device frontend. Single self-contained file: HTML, CSS, and JavaScript all inline, no build step. Includes a baked-in DEFAULT_API_URL constant.	Edit the file, commit, and git push to the GitHub repo. Render auto-redeploys on push.
images/	PNG artwork for the seeded runewords, referenced as images/{id}.png when a Runewords row has no uploaded photo.	Add/replace files in this folder in the repo and push, same as index.html.
Code.gs	The backend. Lives inside a Google Apps Script project bound to the teacher's Google Sheet.	Edit the code in the Apps Script editor, then Deploy → Manage deployments → pencil icon → Version: New version → Deploy. Editing alone does NOT affect the live app. Editing an existing deployment keeps the same /exec URL; always prefer this over creating a new deployment.
runic-system.html	A legacy, fully offline fallback version that stores everything in browser localStorage instead of the Sheet. No multi-device sync, no Runewords tab, no Manage Items, no award/transform history.	Standalone; unrelated to everything above. Only touch if asked.

This revision's setup() also creates two new sheet tabs (RuneAwards, Transforms) the first time it's run against a Sheet that doesn't already have them — existing Students/Pins/Config/Runewords data is never touched by running setup() again.

9.2 Rules to respect when changing game content

- **Runewords** still only need edits through Manage Items or the Runewords sheet directly — never hardcoded back into index.html or Code.gs.
- **The twelve base runes** are still the one exception that stays duplicated between index.html and Code.gs. Any change to a rune itself (not a runeword) needs both files edited identically.
- **RuneAwards and Transforms are append-only logs.** Nothing in the app currently edits or deletes an individual row on either tab (aside from teacherReset clearing everything). If you're asked to add an “undo” for a specific award or Transform, that's new functionality, not an existing one — design and log it deliberately rather than quietly deleting history rows.
- **Keep Transform logging inside apiTransform itself**, not as a separate call the frontend has to remember to make. It's deliberately built this way so every Transform is captured by construction; moving the logging call out to the frontend (or to a second, skippable backend step) would make it possible for a future edit to bypass it.

- After any Code.gs edit, remind whoever is deploying that a new deployment version is required (Section 9.1).
- Never move game-rule validation (roll odds, transform math, the God Rune recipe, craft requirements, what counts as a legal rune award) into the frontend only. The backend must remain the source of truth.
- The Sheet's column headers are load-bearing across every helper that looks columns up by header name, not position. Renaming a header breaks the mapping; adding a new header is safe.
- If you add a new Teacher-tab panel, follow the subtab pattern in Section 6.7 rather than adding it as another always-visible panel, unless there's a specific reason a teacher needs to see it constantly.

9.3 What NOT to do

- Do not publish index.html through Claude's Artifact tool as the “live” version — Claude's hosted artifact preview blocks outbound network calls to external domains like script.google.com, so the backend connection will silently fail there. This file must be opened as a real file (pushed to GitHub → served by Render, or opened locally) to function. (A standalone mockup with fake/local data, built solely to preview a visual or UI change, is fine and was used repeatedly while building the features in this revision — the restriction is specifically on publishing the real, backend-connected index.html.)
- Do not store the real 4-digit PIN or the teacher passphrase anywhere except as intended.
- Do not remove the LockService locks around mutating Code.gs functions, including apiTeacherAwardRune and the Transform-logging path inside apiTransform.
- Do not raise MAX_IMAGE_CHARS casually to “fix” an image upload failure (runeword photos or the site logo) — it exists because a single Google Sheets cell has a hard ~50,000-character ceiling.
- Do not assume “Reset All Data” only touches Students/Pins/PowerWordRedemptions any more — it now also clears RuneAwards and Transforms. If a future change needs a reset that preserves history, that's a new, separate action to design, not a tweak to the existing one.

9.4 Good first questions to ask the teacher before changing something

- Is this a runeword content change? That's a Manage Items task in the app itself.
- Is this a base-rune change or a behavior change? Base-rune changes need edits in two files; behavior changes are usually one file.
- Does this change need to preserve existing student data, the runeword catalog, or the award/transform history logs? (Almost always yes.)
- Will the teacher need to redeploy the Apps Script, and do they still remember how (Section 9.1)?
- If this adds a new Teacher-tab panel: does it belong always-visible, or collapsed behind the subtab strip (Section 6.7)? Default to collapsed unless there's a clear reason otherwise.
- If a Code.gs change is needed, will it require a new deployment or has a new deployment already been created with a different URL that needs to be re-baked into index.html's DEFAULT_API_URL?

9.5 Known limitations, by design

Limitation	Why it's acceptable here
Google Sheets isn't built for heavy concurrent traffic.	Fine for a class of dozens acting occasionally; would need a real database if this ever scaled to hundreds of simultaneous users.
The teacher passphrase and student names travel as plain JSON over HTTPS.	Encrypted in transit by HTTPS; acceptable for a fun in-class tool with no sensitive personal data.
Any two people can independently create an account under the same name	if the second person gets there before the first sets a PIN. First to set a PIN "owns" that name from then on.
No true user accounts.	Deliberately avoided so any student can start playing immediately with just a name.
A single deleted or renamed image file in <code>images/</code> silently breaks a seeded item's photo.	There's no validation that <code>images/{id}.png</code> exists. Acceptable since it's cosmetic.
RuneAwards and Transforms grow forever with no archiving or pagination.	Fine for one class's worth of a school year; a teacher wanting to start fresh has "Reset All Data," which now clears both logs too.

10. One-paragraph mental model

If you remember only one thing from this document:

The webpage is just a display. Every rule — who can act, what a roll produces, what a craft costs, what a God Rune requires, what counts as a legal Transform, what a teacher is allowed to hand out directly — lives in `Code.gs`, which is the only thing allowed to read or write the Google Sheet. The Sheet is the entire database: student data, the runeword catalog, and now also a growing, automatically-written history of every rune awarded and every Transform performed. Nothing is real until `Code.gs` says it's real — and on the Teacher tab, nothing about a collapsed panel is even fetched until a teacher actually clicks it open.

Document generated for the teacher's ongoing use with Claude. Keep this alongside `index.html`, the `images/` folder, and `Code.gs`, and share it at the start of any future conversation asking Claude to modify the Runic System.